

NPC(ロボット)の追加

新しいNPC、ロボットを追加していきます。

ロボットのモデルデータについては、Teamsよりダウンロードしてください。



```
EnemyBase.h
```

```
class EnemyBase : public CharactorBase
{
```

```
public:
```

```
    // 種別
    enum class TYPE
    {
        RAT,
        ROBOT,
    };
```

```
EnemyRobot.h
```

```
#pragma once
#include <DxLib.h>
#include "EnemyBase.h"
```

```
class EnemyRobot : public EnemyBase
{
```

```
public:
```

```
    // 状態
    enum class STATE
```

```

{
    NONE,
    THINK,
    IDLE,
    PATROL,
    SURPRISE,
    ALERT,
    CHASE,
    ATTACK_KICK,
    ATTACK_SHOOT,
    ESCAPE,
    DEAD,
    KNOCKBACK,
    END
};

// アニメーション種別
enum class ANIM_TYPE
{
    DANCE = 0,
    DIE = 1,
    HIT = 3,
    IDLE = 5,
    KICK = 9,
    RUN = 12,
    SHOOT = 13,
    WALK = 15,
};

// コンストラクタ
EnemyRobot(const EnemyBase::EnemyData& data);

// デストラクタ
~EnemyRobot(void) override;

protected:

// リソースロード
void InitLoad(void) override;

// 大きさ、回転、座標の初期化

```

```

void InitTransform(void) override;

// 衝突判定の初期化
void InitCollider(void) override;

// アニメーションの初期化
void InitAnimation(void) override;

// 初期化後の個別処理
void InitPost(void) override;

// 更新系
void UpdateProcess(void) override;
void UpdateProcessPost(void) override;

private:

// モデルの大きさ
static constexpr float SCALE = 0.5f;

// モデルの回転調整
static constexpr VECTOR DEFAULT_LOCAL_ROT =
{ 0.0f, 180.0f * DX_PI_F / 180.0f, 0.0f };

// 衝突判定用線分開始
static constexpr VECTOR COL_LINE_START_LOCAL_POS =
{ 0.0f, 80.0f, 0.0f };

// 衝突判定用線分終了
static constexpr VECTOR COL_LINE_END_LOCAL_POS =
{ 0.0f, -10.0f, 0.0f };

// 衝突判定(移動範囲)用線分開始
static constexpr VECTOR COL_LINE_START_LOCAL_MOVE_POS =
{ 0.0f, 80.0f, 400.0f };

// 衝突判定(移動範囲)用線分終了
static constexpr VECTOR COL_LINE_END_LOCAL_MOVE_POS =
{ 0.0f, -10.0f, 400.0f };

// 衝突判定用カプセル上部球体

```

```
static constexpr VECTOR COL_CAPSULE_TOP_LOCAL_POS =  
{ 0.0f, 40.0f, 80.0f };
```

```
// 衝突判定用カプセル下部球体
```

```
static constexpr VECTOR COL_CAPSULE_DOWN_LOCAL_POS =  
{ 0.0f, 40.0f, -40.0f };
```

```
// 衝突判定用カプセル球体半径
```

```
static constexpr float COL_CAPSULE_RADIUS = 30.0f;
```

```
// 状態
```

```
STATE state_;
```

```
// 更新ステップ
```

```
float step_;
```

```
// 状態遷移
```

```
void ChangeState(STATE state);
```

```
void ChangeStateNone(void);
```

```
void ChangeStateThink(void);
```

```
void ChangeStateIdle(void);
```

```
void ChangeStatePatrol(void);
```

```
void ChangeStateSurprise(void);
```

```
void ChangeStateAlert(void);
```

```
void ChangeStateChase(void);
```

```
void ChangeStateAttackKick(void);
```

```
void ChangeStateAttackShoot(void);
```

```
void ChangeStateEscape(void);
```

```
void ChangeStateDead(void);
```

```
void ChangeStateKnockBack(void);
```

```
void ChangeStateEnd(void);
```

```
// 更新系
```

```
void UpdateNone(void);
```

```
void UpdateThink(void);
```

```
void UpdateIdle(void);
```

```
void UpdatePatrol(void);
```

```
void UpdateSurprise(void);
```

```
void UpdateAlert(void);
```

```
void UpdateChase(void);
```

```
void UpdateAttackKick(void);
```

```

    void UpdateAttackShoot(void);
    void UpdateEscape(void);
    void UpdateDead(void);
    void UpdateKnockBack(void);
    void UpdateEnd(void);

};

```

EnemyRobot.cpp

```

#include "../.../Utility/AsoUtility.h"
#include "../.../Manager/ResourceManager.h"
#include "../.../Manager/SceneManager.h"
#include "../.../Common/AnimationController.h"
#include "../.../Collider/ColliderLine.h"
#include "../.../Collider/ColliderCapsule.h"
#include "EnemyRobot.h"

EnemyRobot::EnemyRobot(const EnemyBase::EnemyData& data)
:
    EnemyBase(data),
    state_(STATE::NONE),
    step_(0.0f)
{
}

EnemyRobot::~EnemyRobot(void)
{
}

void EnemyRobot::InitLoad(void)
{
    // 基底クラスのリソースロード
    CharactorBase::InitLoad();

    // モデルのロード
    transform_.SetModel(
        resMng_.LoadModelDuplicate(ResourceManager::SRC::ENEMY_ROBOT));
}

```

```

void EnemyRobot::InitTransform(void)
{
    transform_.scl = VScale(AsoUtility::VECTOR_ONE, SCALE);
    transform_.quaRot = Quaternion();
    transform_.quaRotLocal = Quaternion::Euler(DEFAULT_LOCAL_ROT);
    transform_.Update();
}

void EnemyRobot::InitCollider(void)
{
    // 主に地面との衝突で仕様する線分コライダ
    ColliderLine* colLine = new ColliderLine(
        ColliderBase::TAG::ENEMY, &transform_,
        COL_LINE_START_LOCAL_POS, COL_LINE_END_LOCAL_POS);
    ownColliders_.emplace(
        static_cast<int>(COLLIDER_TYPE::LINE), colLine);

    // 主に壁や木などの衝突で仕様するカプセルコライダ
    ColliderCapsule* colCapsule = new ColliderCapsule(
        ColliderBase::TAG::ENEMY, &transform_,
        COL_CAPSULE_TOP_LOCAL_POS,
        COL_CAPSULE_DOWN_LOCAL_POS, COL_CAPSULE_RADIUS);
    ownColliders_.emplace(
        static_cast<int>(COLLIDER_TYPE::CAPSULE), colCapsule);
}

void EnemyRobot::InitAnimation(void)
{
    animationController_ = new AnimationController(transform_.modelId);

    // FBX内のアニメーション設定
    int type = -1;

    type = static_cast<int>(ANIM_TYPE::DANCE);
    animationController_>AddInFbx(type, 10.0f, type);
}

```

```

type = static_cast<int>(ANIM_TYPE::IDLE);
animationController_>AddInFbx(type, 20.0f, type);

type = static_cast<int>(ANIM_TYPE::WALK);
animationController_>AddInFbx(type, 30.0f, type);

type = static_cast<int>(ANIM_TYPE::RUN);
animationController_>AddInFbx(type, 30.0f, type);

type = static_cast<int>(ANIM_TYPE::KICK);
animationController_>AddInFbx(type, 45.0f, type);

type = static_cast<int>(ANIM_TYPE::SHOOT);
animationController_>AddInFbx(type, 30.0f, type);

// 初期アニメーション再生
animationController_>Play(static_cast<int>(ANIM_TYPE::IDLE), true);
}

void EnemyRobot::InitPost(void)
{

// 状態遷移初期処理登録
stateChanges_.emplace(static_cast<int>(STATE::NONE),
    std::bind(&EnemyRobot::ChangeStateNone, this));
stateChanges_.emplace(static_cast<int>(STATE::THINK),
    std::bind(&EnemyRobot::ChangeStateThink, this));
stateChanges_.emplace(static_cast<int>(STATE::IDLE),
    std::bind(&EnemyRobot::ChangeStateIdle, this));
stateChanges_.emplace(static_cast<int>(STATE::PATROL),
    std::bind(&EnemyRobot::ChangeStatePatrol, this));
stateChanges_.emplace(static_cast<int>(STATE::SURPRISE),
    std::bind(&EnemyRobot::ChangeStateSurprise, this));
stateChanges_.emplace(static_cast<int>(STATE::ALERT),
    std::bind(&EnemyRobot::ChangeStateAlert, this));
stateChanges_.emplace(static_cast<int>(STATE::CHASE),
    std::bind(&EnemyRobot::ChangeStateChase, this));
stateChanges_.emplace(static_cast<int>(STATE::ATTACK_KICK),
    std::bind(&EnemyRobot::ChangeStateAttackKick, this));
stateChanges_.emplace(static_cast<int>(STATE::ATTACK_SHOOT),

```

```

        std::bind(&EnemyRobot::ChangeStateAttackShoot, this));
stateChanges_.emplace(static_cast<int>(STATE::ESCAPE),
    std::bind(&EnemyRobot::ChangeStateEscape, this));
stateChanges_.emplace(static_cast<int>(STATE::DEAD),
    std::bind(&EnemyRobot::ChangeStateDead, this));
stateChanges_.emplace(static_cast<int>(STATE::KNOCKBACK),
    std::bind(&EnemyRobot::ChangeStateKnockBack, this));
stateChanges_.emplace(static_cast<int>(STATE::END),
    std::bind(&EnemyRobot::ChangeStateEnd, this));

// 初期状態設定
ChangeState(STATE::THINK);

}

void EnemyRobot::UpdateProcess(void)
{
    // 状態別更新
    stateUpdate_();
}

void EnemyRobot::UpdateProcessPost(void)
{
    EnemyBase::UpdateProcessPost();
}

void EnemyRobot::ChangeState(STATE state)
{
    state_ = state;
    EnemyBase::ChangeState(static_cast<int>(state_));
}

void EnemyRobot::ChangeStateNone(void)
{
    stateUpdate_ = std::bind(&EnemyRobot::UpdateNone, this);
}

void EnemyRobot::ChangeStateThink(void)
{

```



```

stateUpdate_ = std::bind(&EnemyRobot::UpdateThink, this);

// 思考：最初はIDLEのみ
ChangeState (STATE::IDLE);

}

void EnemyRobot::ChangeStateIdle(void)
{

stateUpdate_ = std::bind(&EnemyRobot::UpdateIdle, this);

// ランダムな待機時間
step_ = 3.0f + static_cast<float>(GetRand(3));

// 移動量ゼロ
movePow_ = AsoUtility::VECTOR_ZERO;

// 待機アニメーション再生
animationController_>Play(
    static_cast<int>(ANIM_TYPE::IDLE), true);

}

void EnemyRobot::ChangeStatePatrol(void)
{
stateUpdate_ = std::bind(&EnemyRobot::UpdatePatrol, this);
}

void EnemyRobot::ChangeStateSurprise(void)
{
stateUpdate_ = std::bind(&EnemyRobot::UpdateSurprise, this);
}

void EnemyRobot::ChangeStateAlert(void)
{
stateUpdate_ = std::bind(&EnemyRobot::UpdateAlert, this);
}

void EnemyRobot::ChangeStateChase(void)
{

```

```

    stateUpdate_ = std::bind(&EnemyRobot::UpdateChase, this);
}

void EnemyRobot::ChangeStateAttackKick(void)
{
    stateUpdate_ = std::bind(&EnemyRobot::UpdateAttackKick, this);
}

void EnemyRobot::ChangeStateAttackShoot(void)
{
    stateUpdate_ = std::bind(&EnemyRobot::UpdateAttackShoot, this);
}

void EnemyRobot::ChangeStateEscape(void)
{
    stateUpdate_ = std::bind(&EnemyRobot::UpdateEscape, this);
}

void EnemyRobot::ChangeStateDead(void)
{
    stateUpdate_ = std::bind(&EnemyRobot::UpdateDead, this);
}

void EnemyRobot::ChangeStateKnockBack(void)
{
    stateUpdate_ = std::bind(&EnemyRobot::UpdateKnockBack, this);
}

void EnemyRobot::ChangeStateEnd(void)
{
    stateUpdate_ = std::bind(&EnemyRobot::UpdateEnd, this);
}

void EnemyRobot::UpdateNone(void)
{
}

void EnemyRobot::UpdateThink(void)
{
}

```

```

void EnemyRobot::UpdateIdle(void)
{

    step_ -= scnMng_.GetDeltaTime();
    if (step_ < 0.0f)
    {
        // 待機終了
        ChangeState(STATE::THINK);
        return;
    }

}

void EnemyRobot::UpdatePatrol(void)
{
}

void EnemyRobot::UpdateSurprise(void)
{
}

void EnemyRobot::UpdateAlert(void)
{
}

void EnemyRobot::UpdateChase(void)
{
}

void EnemyRobot::UpdateAttackKick(void)
{
}

void EnemyRobot::UpdateAttackShoot(void)
{
}

void EnemyRobot::UpdateEscape(void)
{
}

```

```

void EnemyRobot::UpdateDead(void)
{
}

void EnemyRobot::UpdateKnockBack(void)
{
}

void EnemyRobot::UpdateEnd(void)
{
}

```

【要件】

外部ファイルにロボット用のデータを追加して、
画面にロボットが表示されるようになること。

- ・ ResourceManagerにロボットのモデルデータを追加すること
- ・ EnemyBaseにロボット種別を追加すること
- ・ EnemyManagerにロボットのインスタンス生成機能を追加すること
- ・ 外部ファイルにロボット用のデータを追加すること

ID	敵種別	HP	初期座標X	初期座標Y	初期座標Z	移動可能範囲
1	0	2	299.52f	207.43f	1885.02f	1000.0f
2	0	2	-71.29f	12.66f	1673.86f	1500.0f
3	0	2	828.97f	69.91f	1559.43f	2000.0f
4	1	3	1828.97f	69.91f	0.0f	-1.0f

このあと、ロボットの行動を作るにあたり、
広い場所が好ましいため、座標は上記に合わせるようにしてください。

【目標】

指定座標にロボットモデルが描画されていること。

